Machine Coding: Build a Calculator

Problem: Build a calculator UI (digits, decimal, + − × ÷, equals, clear, backspace, ±, %) with keyboard support
Tech stack: React 19 + TypeScript (Vite), plain CSS
Time limit: 60 minutes

PART A: Thought process

The general steps for any machine coding task:

1. **Understand the requirements.** Restate the problem, split functional and non-functional needs, ask clarifying questions, write down assumptions.
2. **List the interactions.** Every button / key and what it should do. This table becomes your checklist.
3. **Design the architecture.** Few components, one responsibility each. Start simple.
4. **Design the state.** The smallest state that can describe every screen. Derive the rest.
5. **Design data shapes and contracts.** Props, action types, example data.
6. **Trace one data flow end to end.** Proves the design before you code.
7. **Find the hard parts.** Solve them on paper first, they decide the design.
8. **Build in layers.** Simplest working version first, then add one working layer at a time.
9. **Cover states and edge cases.** Initial, typing, result, error, and the weird inputs.
10. **Wrap up.** Trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements, questions, use cases	5-7 min
Architecture + state + hard parts on paper	8-10 min
Static UI (display + keypad grid)	8 min
Core logic (reducer + evaluator)	20 min
Keyboard, edge cases, error state	8 min
Styling, accessibility, testing by hand	5 min
Wrap-up and trade-offs	2-3 min

Tip: think out loud. The interviewer grades the reasoning as much as the result. Say "I'll keep the input as tokens instead of one string because...", "I'm not using `eval` because...". If you get stuck, say what you are trying, and the interviewer can nudge you.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Digits 0-9 and a decimal point build a number.
- Operators + − × ÷ between numbers.
- `=` computes the result using **normal math precedence** (2 + 3 × 4 = 14).
- `AC` clears everything, `⌫` deletes the last character.
- `±` flips the sign of the current number, `%` divides it by 100.
- The display shows the expression being typed and a live preview of the result.
- Keyboard support: digits, `. + - * / % Enter = Backspace Escape`.
- Division by zero shows an error instead of `Infinity`.

Non-functional requirements

Area	What it means here
Correctness	No `eval`, precedence handled, floating point noise removed (0.1 + 0.2 = 0.3)
Performance	Trivial data, but no listeners leaking and no re-attaching the keydown listener on every render
Error handling	Divide by zero, overflow (`Infinity`), invalid sequences (`5 + × =`) never crash
Accessibility	Real `<button>`s, `aria-label` for symbols (÷, ⌫, ±), visible focus ring, display is `aria-live`
Usability	Keyboard works, layout does not jump, long numbers wrap
Reusability / extensibility	Keys are a config array, logic is a pure reducer: adding `√` or `( )` touches data + one case

Clarifying questions to ask

1. Should `2 + 3 × 4` give **14** (math precedence) or **20** (simple pocket calculator that computes as you go)?
2. Do we need brackets or scientific functions (√, x^2, sin)? Or just the four operators?
3. After `=`, what should a digit do (start fresh) and what should an operator do (continue from result)?
4. How should errors show: a message on the display, or disable keys?
5. Is keyboard support required? Do we need history / memory keys (M+, MR)?
6. How many digits do we support? What happens when a number is too long?

Assumptions we build with

- Math precedence (× ÷ before + −), evaluated left to right. No brackets.
- After `=`: a digit or `.` starts a new calculation; an operator continues from the result.
- Two operators in a row: the second **replaces** the first (`5 + ×` → `5 ×`).
- `=` with a dangling operator ignores it (`5 + =` → `5`).
- Max 15 digits per number (JS `Number` precision limit).

- Results rounded to 12 significant digits to hide float noise.
- Division by zero shows "Cannot divide by zero"; next digit starts fresh.
- No history, no memory keys, no persistence.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Press `7`, `8`	Display `78`
Press `0`, `0`, `7`	Display `7` (no leading zeros)
Press `.` on empty	Display `0.`
Press `1`, `.`, `.`, `2`	Display `1.2` (second dot ignored)
Press `2 + 3`	Display `2 + 3`, preview `= 5`
Press `2 + 3 × 4 =`	Display `14` (precedence)
Press `10 − 2 − 3 =`	`5` (left to right, not 10 − (2 − 3))
Press `5 + ×`	Display `5 ×` (operator replaced)
Press `5 + =`	`5` (dangling operator dropped)
Press `0.1 + 0.2 =`	`0.3`, not `0.30000000000000004`
Press `5 ÷ 0 =`	Error "Cannot divide by zero" in red
After error, press `7`	Error cleared, display `7`
After error, press `+`	Ignored
After `2 + 3 =`, press `4`	New calculation, display `4`
After `2 + 3 =`, press `× 2 =`	`10` (continue from result)
Press `12 ⌫`	`1`
Press `⌫` after a result	Clears
Press `5 ±`	`-5`; press again → `5`
Press `50 %`	`0.5`
Press `+` on empty	`0 +`
Type 16 digits	Stops at 15
Keyboard `1 0 / 4 Enter`	`2.5`
Focus a button, press Enter	Only `=` runs, the focused button does **not** also click
`Ctrl+C`, `Ctrl+R`	Still work (not swallowed)
`Escape`	Clear

---

3. Design the architecture

Module	Responsibility
`utils/calc.ts`	Pure math: `evaluate(tokens)` with precedence, `formatNumber(n)` rounding, `isOperator`
`calculatorReducer.ts`	Pure state machine: (state, action) → next state. All input rules live here
`components/Calculator.tsx`	Owns state (`useReducer`), keyboard listener, derives display strings
`components/Display.tsx`	Shows expression / error and live preview. Stateless
`components/Keypad.tsx`	Renders the button grid from a `KEYS` config array, calls `onPress(action)`
`App.tsx`	Mounts the calculator

```
App
└── Calculator            (useReducer + keydown listener)
    ├── Display           (expression, preview, error)   ← props
    └── Keypad            (KEYS config → buttons)        → onPress(action) = dispatch
          │
          ▼
    calculatorReducer ──uses──▶ utils/calc (evaluate, formatNumber)
```

Why this split: buttons and keyboard both produce the **same** `Action` objects, so there is one code path for input. The reducer and math are pure functions, so they are testable in Node without React. Start with everything in `Calculator` ; pull the reducer out the moment the switch grows past 2-3 cases.

---

4. Design the state / data model

State or field	Purpose
`tokens: string[]`	The expression as tokens, e.g. `['12', '+', '3.5']` . The last number token is the one being typed
`evaluated: boolean`	True right after `=` ; tells the next digit to start a new calculation
`error: string | null`	Error message to show (divide by zero, overflow)

```
export type State = {
  tokens: string[]
  evaluated: boolean
  error: string | null
}

export const initialState: State = { tokens: [], evaluated: false, error: null }
```

Why tokens and not one string like `"12+3.5"`? With a string, every rule needs parsing: "is the last char an operator?", "does the current number already have a dot?", "is this `-` a minus or a negative sign?". With tokens the last number is just `tokens.at(-1)`, and a negative number `'-5'` can never be confused with the operator `'−'` (we use the Unicode minus for the operator).

Why numbers stay strings while typing: `Number('3.')` is `3` and `Number('0.0')` is `0`. If we stored numbers, the user could never type `3.05`. Convert only when evaluating.

Senior note: derive, don't store

- `expression` = `tokens.join(' ') || '0'` → derived.
- `preview` (live result) = `evaluate(tokens)` while typing → derived, recomputed each render (cheap).
- `currentNumber`, `lastIsOperator`, `hasDot` → all derived from `tokens.at(-1)`.
- Storing any of these would create two sources of truth that can disagree.

Who owns state: only `Calculator`. `Display` and `Keypad` are stateless and get props. `dispatch` is passed down as `onPress` (it is stable, so it never causes extra effects).

Why `useReducer` over several `useState`s: each key changes several fields together (`=` sets tokens, evaluated and error at once). A reducer keeps every transition in one place and makes "impossible states" (error + evaluated) easy to avoid.

---

5. Design the data structures and contracts

Example states

```
// typing "2 + 3 × 4"
{ tokens: ['2', '+', '3', '×', '4'], evaluated: false, error: null }   // preview "= 14"

// after "="
{ tokens: ['14'], evaluated: true, error: null }

// after "5 ÷ 0 ="
{ tokens: [], evaluated: false, error: 'Cannot divide by zero' }
```

Tokens always **alternate** number, operator, number... The reducer guarantees it (operators replace operators, numbers extend numbers), so `evaluate` can walk the array two at a time without validation. No ids needed: the order of tokens is the identity, and the key buttons use their unique `label` as React `key`.

Action contract (the only way to change state)

```
export type Action =
  | { type: 'digit'; digit: string }
  | { type: 'dot' }
  | { type: 'operator'; op: Operator }   // '+' | '−' | '×' | '÷'
  | { type: 'equals' }
  | { type: 'clear' }
  | { type: 'backspace' }
  | { type: 'toggleSign' }
  | { type: 'percent' }
```

Key config contract (Keypad)

```
type Key = {
  label: string                    // what the button shows: '÷'
  action: Action                   // what it dispatches
  ariaLabel?: string               // what a screen reader says: 'divide'
  variant?: 'fn' | 'op' | 'eq'     // styling only
}
```

Component props

Component	Props in	Out
`Display`	`expression: string`, `preview: string`, `error: string | null`	—
`Keypad`	`onPress: (action: Action) => void`	calls `onPress` on click

Pure function contracts

Function	Input	Output
`evaluate(tokens)`	`['2','+','3','×','4']`	`14`; throws `Error('Cannot divide by zero')`
`formatNumber(n)`	`0.30000000000000004`	`'0.3'`; throws on `Infinity` / `NaN`

There is no API; nothing to mock.

---

6. Trace the data flow

Interaction: tokens are `['2', '+', '3', '×', '4']` and the user presses `=` (button or Enter).

```
User clicks "=" (or presses Enter → keydown listener → preventDefault)
        │
        ▼
dispatch({ type: 'equals' })
        │
        ▼
calculatorReducer
  last = '4' → is a number, so no dangling operator to drop
  evaluate(['2','+','3','×','4'])
     pass 1 (× ÷):  nums [2, 3] → 3 × 4 → nums [2, 12], ops ['+']
     pass 2 (+ −):  2 + 12 = 14
  formatNumber(14) → '14'
  returns { tokens: ['14'], evaluated: true, error: null }
        │
        ▼
React re-renders Calculator
  expression = '14'
  preview    = '' (evaluated)
        │
        ▼
Display shows "14"; aria-live announces it to screen readers
Next digit → reducer sees evaluated → starts fresh
Next operator → continues: ['14', '×']
```

---

7. Important technical challenges

Challenge 1: Evaluating with precedence without `eval`

Problem: `2 + 3 × 4` must be 14. The easy trick `eval('2+3*4')` is a security risk (code injection if input ever comes from outside), is blocked by strict CSP, and breaks on our symbols (× ÷ −). Computing as you go gives 20.

Solution: two passes over the alternating tokens. Pass 1 collapses × and ÷ into the previous number. Pass 2 does + and − left to right. That's precedence for two levels with no parser. (For brackets you'd move to shunting-yard, see Q&A.)

```
export function evaluate(tokens: string[]): number {
  const nums = [Number(tokens[0])]
  const ops: string[] = []
  for (let i = 1; i < tokens.length; i += 2) {
    const op = tokens[i]
    const n = Number(tokens[i + 1])
    if (op === '×') nums.push(nums.pop()! * n)
    else if (op === '÷') {
      if (n === 0) throw new Error('Cannot divide by zero')
      nums.push(nums.pop()! / n)
    } else { ops.push(op); nums.push(n) }
  }
  return ops.reduce((acc, op, i) => (op === '+' ? acc + nums[i + 1] : acc - nums[i + 1]), nums[0])
}
```

Challenge 2: Floating point noise

Problem: `0.1 + 0.2` is `0.30000000000000004` and `1.1 × 3` is `3.3000000000000003` in JavaScript (IEEE 754 doubles). A calculator showing that looks broken. Also `1 ÷ 3` would show 16 digits and overflow the display.

Solution: round only at the **output**, to 12 significant digits, then let `parseFloat` drop trailing zeros. Keep full precision in between, so errors don't add up. Very large / small values automatically become `1e+21` style.

```
export function formatNumber(n: number): string {
  if (!Number.isFinite(n)) throw new Error('Number too large')
  return String(parseFloat(n.toPrecision(12)))   // 0.30000000000000004 → "0.3"
}
```

Challenge 3: Input rules (the state machine)

Problem: users press keys in any order. Scenarios that break naive code: `1..2` (two dots), `007` (leading zeros), `5 + × 3` (two operators), `5 + =` (dangling operator), typing a digit right after `=` (should start fresh, not append to the result), typing after an error.

Solution: every rule is one early return in the reducer, based on `last` token and two flags. The reducer is pure, so each rule is a one-line test.

```
const last = tokens.at(-1)
const lastIsNumber = last !== undefined && !isOperator(last)
const fresh = state.evaluated || state.error !== null

case 'digit':
  if (fresh) return { ...initialState, tokens: [action.digit] }        // new calculation
  if (!lastIsNumber) return { ...state, tokens: [...tokens, action.digit] }
  if (last === '0' || last === '-0') return { ...state, tokens: withLast(tokens, last.slice(0, -1) +
action.digit) }
  if (last.replace(/\D/g, '').length >= MAX_DIGITS) return state
  return { ...state, tokens: withLast(tokens, last + action.digit) }

case 'operator':
  if (state.error) return state
  if (last === undefined) return { ...state, tokens: ['0', action.op] }
  if (!lastIsNumber) return { ...state, tokens: withLast(tokens, action.op) }   // replace
  return { ...state, evaluated: false, tokens: [...withLast(tokens, last.replace(/\.$/, '')),
action.op] }
```

Challenge 4: Keyboard support without double input or leaks

Problem: three traps.

1. The user clicks `7`, the button keeps focus, then presses Enter. The browser fires a **click on the focused button** *and* our keydown listener sees Enter → `7` is typed and `=` runs.
2. `/` opens quick find in Firefox.
3. Adding the listener on every render (or forgetting cleanup) leaks listeners and fires each key several times; in StrictMode effects run twice in dev, so missing cleanup shows up at once.

Solution: one listener on `window` in a `useEffect` with `[]` and a cleanup. Map keys to the **same actions** the buttons use. Call `preventDefault()` only for keys we handle, and skip Ctrl/Meta/Alt combos so browser shortcuts still work. `dispatch` is stable, so the empty dependency array is correct (no stale closure).

```
useEffect(() => {
  function onKeyDown(e: KeyboardEvent) {
    if (e.ctrlKey || e.metaKey || e.altKey) return
    const action: Action | undefined = /^\d$/.test(e.key) ? { type: 'digit', digit: e.key } :
KEY_MAP[e.key]
    if (!action) return
    e.preventDefault()   // stops the focused button's click on Enter, and Firefox quick find on "/"
    dispatch(action)
  }
  window.addEventListener('keydown', onKeyDown)
  return () => window.removeEventListener('keydown', onKeyDown)
}, [])
```

---

8. Implement in the correct order

#	Step	Working result after the step
1	Static layout: `Calculator` with a display div and a 4-column CSS grid of buttons	Looks like a calculator, does nothing
2	Move buttons into a `KEYS` config array, extract `Keypad`	Same UI, layout is data
3	Define `State`, `Action`, `useReducer` with `digit` and `clear`	Can type numbers and clear
4	Add `operator` + `Display` showing `tokens.join(' ')`	Can type `12 + 3`
5	Write `evaluate` (two-pass) + `equals`	`2 + 3 × 4 = 14` works
6	Add `formatNumber` (float rounding + `Infinity` check), divide-by-zero error state	`0.1 + 0.2 = 0.3`, `5 ÷ 0` shows error
7	Rules: `dot`, leading zero, operator replace, dangling operator, fresh after `=` / error	Every row of the use-case table passes
8	`backspace`, `toggleSign`, `percent`, max digits	All keys work
9	Live preview (derived in render)	`= 14` shows while typing
10	Keyboard listener with cleanup and `preventDefault`	Full keyboard use
11	Accessibility: `aria-label`s, `aria-live`, focus ring; styling	Usable by screen reader and keyboard
12	Quick tests on the reducer (pure, no DOM)	Confidence for the rules

---

9. Cover all states and edge cases

State / edge case	What the user sees	How it's handled
Initial	`0`	`tokens: []` → `join` gives `''` → fallback `'0'`
Typing a number	`125`	last token extended
Typing an expression	`2 + 3 × 4`, preview `= 14`	preview derived only when ≥ 3 tokens and last is a number
Result	`14`, no preview	`evaluated: true`
Error	red "Cannot divide by zero"	`error` set, tokens cleared; digit starts fresh; operator / % / ± ignored
Empty: operator first	`0 +`	`last === undefined` → `['0', op]`
Empty: `=` or ⌫	`0`	no-op
Two dots	`1.2`	`last.includes('.')` → ignore
Leading zeros	`7` not `007`	`'0'` / `'-0'` replaced
Dot first	`0.5`	push `'0.'`
Trailing dot then operator	`3 +` not `3. +`	strip `.$` before pushing operator
Two operators	`5 ×`	replace last operator
`=` with dangling operator	`5 + =` → `5`	slice off last token
Repeated `=`	stays `14`	`fresh` → no-op
Digit after `=`	starts new	`fresh` → `[digit]`
Operator after `=`	`14 ×`	push op, `evaluated: false`
⌫ on result / error	clears	`fresh` → `initialState`
⌫ on `-5`	removes whole token	regex `^-\d$`
`±` on `0` / result	`-0` / `-14`	string flip; `formatNumber(-0)` prints `0`
`%`	`50` → `0.5`	`formatNumber(n / 100)`
Float noise	`0.3`	`toPrecision(12)`
Huge result (`999999999999999 × 999999999999999`)	`1e+30`	`toPrecision` switches to exponent
Overflow to `Infinity`	"Number too large"	`formatNumber` throws, caught in `equals`
16th digit	ignored	`MAX_DIGITS = 15`
Long expression	wraps to more lines	CSS `overflow-wrap: anywhere`
Ctrl/Cmd shortcuts	still work	listener returns early

State / edge case	What the user sees	How it's handled
Enter on a focused button	only `=`	`preventDefault`

10. Final structure

```
_32_build_calculator/
├── index.html
├── package.json
└── src/
    ├── main.tsx                  (unchanged Vite entry)
    ├── App.tsx                   mounts <Calculator />
    ├── App.css                   calculator styles
    ├── index.css                 page reset / centering
    ├── calculatorReducer.ts      State, Action, initialState, reducer (all input rules)
    ├── utils/
    │   └── calc.ts               evaluate, formatNumber, isOperator, OPERATORS
    └── components/
        ├── Calculator.tsx        useReducer + keyboard + derived display
        ├── Display.tsx           expression / error + preview
        └── Keypad.tsx            KEYS config → buttons
```

11. Full working code

`src/utils/calc.ts`

```
export const OPERATORS = ['+', '−', '×', '÷'] as const
export type Operator = (typeof OPERATORS)[number]

export const isOperator = (token: string): token is Operator =>
  (OPERATORS as readonly string[]).includes(token)

// tokens alternate number, operator, number ... e.g. ['2', '+', '3', '×', '4']
export function evaluate(tokens: string[]): number {
  // pass 1: resolve × and ÷ first (precedence), keep + and − for pass 2
  const nums = [Number(tokens[0])]
  const ops: string[] = []
  for (let i = 1; i < tokens.length; i += 2) {
    const op = tokens[i]
    const n = Number(tokens[i + 1])
    if (op === '×') {
      nums.push(nums.pop()! * n)
    } else if (op === '÷') {
      if (n === 0) throw new Error('Cannot divide by zero')
      nums.push(nums.pop()! / n)
    } else {
      ops.push(op)
      nums.push(n)
    }
  }
  // pass 2: + and − left to right
  return ops.reduce((acc, op, i) => (op === '+' ? acc + nums[i + 1] : acc - nums[i + 1]), nums[0])
}

// 0.1 + 0.2 = 0.30000000000000004 -> round to 12 significant digits -> "0.3"
export function formatNumber(n: number): string {
  if (!Number.isFinite(n)) throw new Error('Number too large')
  return String(parseFloat(n.toPrecision(12)))
}
```

`src/calculatorReducer.ts`

```
import { evaluate, formatNumber, isOperator, type Operator } from './utils/calc.ts'

export type State = {
  tokens: string[] // e.g. ['12', '+', '3.5']; the last number is the one being typed
  evaluated: boolean // true right after "=", so the next digit starts a new calculation
  error: string | null
}

export type Action =
  | { type: 'digit'; digit: string }
  | { type: 'dot' }
  | { type: 'operator'; op: Operator }
  | { type: 'equals' }
  | { type: 'clear' }
  | { type: 'backspace' }
  | { type: 'toggleSign' }
  | { type: 'percent' }

export const initialState: State = { tokens: [], evaluated: false, error: null }

const MAX_DIGITS = 15 // beyond this JS numbers lose precision

// replace the last token (the number being typed)
const withLast = (tokens: string[], last: string) => [...tokens.slice(0, -1), last]

export function calculatorReducer(state: State, action: Action): State {
  const { tokens } = state
  const last = tokens.at(-1)
  const lastIsNumber = last !== undefined && !isOperator(last)
  // after "=" or an error, typing a number starts a new calculation
  const fresh = state.evaluated || state.error !== null

  switch (action.type) {
    case 'digit': {
      if (fresh) return { ...initialState, tokens: [action.digit] }
      if (!lastIsNumber) return { ...state, tokens: [...tokens, action.digit] }
      // no leading zeros: 0 then 7 -> 7, -0 then 7 -> -7
      if (last === '0' || last === '-0') return { ...state, tokens: withLast(tokens, last.slice(0, -1)
+ action.digit) }
      if (last.replace(/\D/g, '').length >= MAX_DIGITS) return state
      return { ...state, tokens: withLast(tokens, last + action.digit) }
    }
    case 'dot': {
      if (fresh) return { ...initialState, tokens: ['0.'] }
      if (!lastIsNumber) return { ...state, tokens: [...tokens, '0.'] }
```

```
      if (last.includes('.')) return state
      return { ...state, tokens: withLast(tokens, last + '.') }
    }
    case 'operator': {
      if (state.error) return state
      if (last === undefined) return { ...state, tokens: ['0', action.op] }
      // a second operator replaces the first: 5 + × -> 5 ×
      if (!lastIsNumber) return { ...state, tokens: withLast(tokens, action.op) }
      // after "=" this continues from the result; "3." is tidied to "3"
      return { ...state, evaluated: false, tokens: [...withLast(tokens, last.replace(/\.$/, '')),
action.op] }
    }
    case 'equals': {
      if (fresh || last === undefined) return state
      const expr = lastIsNumber ? tokens : tokens.slice(0, -1) // drop a dangling operator: 5 + = -> 5
      try {
        return { tokens: [formatNumber(evaluate(expr))], evaluated: true, error: null }
      } catch (e) {
        return { ...initialState, error: (e as Error).message }
      }
    }
    case 'backspace': {
      if (fresh) return initialState
      // remove the whole token when only one digit (or "-digit") is left
      if (!lastIsNumber || last.length === 1 || /^-\d$/.test(last)) return { ...state, tokens:
tokens.slice(0, -1) }
      return { ...state, tokens: withLast(tokens, last.slice(0, -1)) }
    }
    case 'toggleSign': {
      if (!lastIsNumber) return state
      return { ...state, tokens: withLast(tokens, last.startsWith('-') ? last.slice(1) : '-' + last) }
    }
    case 'percent': {
      if (!lastIsNumber) return state
      return { ...state, tokens: withLast(tokens, formatNumber(Number(last) / 100)) }
    }
    case 'clear':
      return initialState
  }
}
```

`src/components/Calculator.tsx`

```
import { useEffect, useReducer } from 'react'
import { calculatorReducer, initialState, type Action } from '../calculatorReducer.ts'
import { evaluate, formatNumber, isOperator } from '../utils/calc.ts'
import Display from './Display.tsx'
import Keypad from './Keypad.tsx'

const KEY_MAP: Record<string, Action> = {
  '+': { type: 'operator', op: '+' },
  '-': { type: 'operator', op: '−' },
  '*': { type: 'operator', op: '×' },
  '/': { type: 'operator', op: '÷' },
  '.': { type: 'dot' },
  '%': { type: 'percent' },
  '=': { type: 'equals' },
  Enter: { type: 'equals' },
  Backspace: { type: 'backspace' },
  Escape: { type: 'clear' },
}

// live result while typing: shown once there is a complete operation, e.g. 2 + 3
function getPreview(tokens: string[]): string {
  if (tokens.length < 3 || isOperator(tokens[tokens.length - 1])) return ''
  try {
    return formatNumber(evaluate(tokens))
  } catch {
    return ''
  }
}

export default function Calculator() {
  const [state, dispatch] = useReducer(calculatorReducer, initialState)

  useEffect(() => {
    function onKeyDown(e: KeyboardEvent) {
      if (e.ctrlKey || e.metaKey || e.altKey) return // keep Ctrl+C, Ctrl+R etc. working
      const action: Action | undefined = /^\d$/.test(e.key) ? { type: 'digit', digit: e.key } :
KEY_MAP[e.key]
      if (!action) return
      // Enter on a focused button would also fire its click (double input); "/" opens quick find in
Firefox
      e.preventDefault()
      dispatch(action)
    }
    window.addEventListener('keydown', onKeyDown)
    return () => window.removeEventListener('keydown', onKeyDown)
```

```
  }, [])

  const expression = state.tokens.join(' ') || '0'
  const preview = state.evaluated ? '' : getPreview(state.tokens)

  return (
    <section className="calculator" aria-label="Calculator">
      <Display expression={expression} preview={preview} error={state.error} />
      <Keypad onPress={dispatch} />
    </section>
  )
}
```

src/components/Display.tsx

```
type Props = {
  expression: string
  preview: string
  error: string | null
}

export default function Display({ expression, preview, error }: Props) {
  return (
    <div className={`display${error ? ' error' : ''}`} aria-live="polite">
      <div className="expression" data-testid="expression">{error ?? expression}</div>
      <div className="preview">{preview && `= ${preview}`}</div>
    </div>
  )
}
```

`src/components/Keypad.tsx`

```
import type { Dispatch } from 'react'
import type { Action } from '../calculatorReducer.ts'

type Key = {
  label: string
  action: Action
  ariaLabel?: string // screen readers read "÷" badly, so symbols get a word
  variant?: 'fn' | 'op' | 'eq'
}

const digit = (d: string): Key => ({ label: d, action: { type: 'digit', digit: d } })

// the layout is data: reordering or adding a key never touches the JSX
const KEYS: Key[] = [
  { label: 'AC', action: { type: 'clear' }, ariaLabel: 'clear', variant: 'fn' },
  { label: '±', action: { type: 'toggleSign' }, ariaLabel: 'toggle sign', variant: 'fn' },
  { label: '%', action: { type: 'percent' }, ariaLabel: 'percent', variant: 'fn' },
  { label: '÷', action: { type: 'operator', op: '÷' }, ariaLabel: 'divide', variant: 'op' },
  digit('7'), digit('8'), digit('9'),
  { label: '×', action: { type: 'operator', op: '×' }, ariaLabel: 'multiply', variant: 'op' },
  digit('4'), digit('5'), digit('6'),
  { label: '−', action: { type: 'operator', op: '−' }, ariaLabel: 'minus', variant: 'op' },
  digit('1'), digit('2'), digit('3'),
  { label: '+', action: { type: 'operator', op: '+' }, ariaLabel: 'plus', variant: 'op' },
  digit('0'),
  { label: '.', action: { type: 'dot' }, ariaLabel: 'decimal point' },
  { label: '⌫', action: { type: 'backspace' }, ariaLabel: 'backspace' },
  { label: '=', action: { type: 'equals' }, ariaLabel: 'equals', variant: 'eq' },
]

export default function Keypad({ onPress }: { onPress: Dispatch<Action> }) {
  return (
    <div className="keypad">
      {KEYS.map((key) => (
        <button
          key={key.label}
          type="button"
          className={`key ${key.variant ?? ''}`}
          aria-label={key.ariaLabel}
          onClick={() => onPress(key.action)}
        >
          {key.label}
        </button>
      ))}
    </div>
```

```
  )
}
```

src/App.tsx

```
import Calculator from './components/Calculator.tsx'
import './App.css'

export default function App() {
  return <Calculator />
}
```

`src/App.css`

```
.calculator {
  width: min(320px, 100% - 32px);
  padding: 16px;
  border-radius: 20px;
  background: #1e293b;
  box-shadow: 0 20px 40px rgb(0 0 0 / 0.25);
}

.display {
  min-height: 96px;
  padding: 8px 8px 16px;
  text-align: right;
  color: #f8fafc;
  overflow-wrap: anywhere; /* long numbers wrap instead of overflowing the card */
}

.expression {
  font-size: 2rem;
  line-height: 1.2;
}

.display.error .expression {
  color: #f87171;
  font-size: 1.4rem;
}

.preview {
  min-height: 1.5em; /* reserve the line so the layout does not jump */
  font-size: 1.1rem;
  color: #94a3b8;
}

.keypad {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 10px;
}

.key {
  height: 60px;
  border: 0;
  border-radius: 14px;
  font: inherit;
  font-size: 1.3rem;
  color: #f8fafc;
```

```
  background: #334155;
  cursor: pointer;
}

.key:hover {
  filter: brightness(1.15);
}

.key:active {
  transform: scale(0.96);
}

.key:focus-visible {
  outline: 3px solid #38bdf8;
  outline-offset: 2px;
}

.key.fn {
  background: #64748b;
}

.key.op {
  background: #f59e0b;
}

.key.eq {
  background: #22c55e;
}
```

`src/index.css`

```
:root {
  font-family: system-ui, Avenir, Helvetica, Arial, sans-serif;
  background: #f1f5f9;
}

body {
  margin: 0;
  min-height: 100vh;
  display: grid;
  place-items: center;
}
```

`src/main.tsx` stays as the Vite template created it.

---

12. Wrap-up

Edge cases handled

- Precedence and left-to-right order (`10 - 2 - 3 = 5`), no `eval`.
- Float noise, huge numbers, `Infinity`, divide by zero.
- Double dot, leading zeros, two operators, dangling operator, repeated `=`.
- Fresh start after a result or error; continue from a result with an operator.
- Keyboard with no double input, no leaked listeners, browser shortcuts untouched.

Trade-offs to mention

Decision	Why	Cost
Tokens array, numbers as strings	Typing `3.05` works; rules read the last token easily	Convert with `Number()` at evaluation
Two-pass evaluator	15 lines, easy to explain, covers 2 precedence levels	No brackets / power; needs shunting-yard for those
Round to 12 significant digits at output	Hides float noise with zero dependencies	Not exact decimal math; money-grade needs a decimal library or integer cents
`useReducer` with pure reducer	All rules in one testable place	A bit more boilerplate than `useState`
Keys as config array	Layout changes are data-only	One more indirection to read
Global `window` keydown	Works without focusing the calculator	If the page has other inputs, it would steal their keys (scope it to the calculator element then)
Preview computed every render	Always in sync, no extra state	Re-evaluates on each key (trivial cost)

With more time

- **Tests:** Vitest for the reducer and `evaluate` (table-driven, like the use-case table); React Testing Library for "click 2 + 3 = shows 5" and the Enter double-fire.
- **Features:** brackets and `^` via shunting-yard, repeat last operation on repeated `=`, history list, memory keys, copy result.
- **UX:** shrink font for long numbers, show which operator is active, thousands separators (`Intl.NumberFormat`).
- **Precision:** `decimal.js` or BigInt-based decimals for exact results.
- **Accessibility:** announce only results (not every keystroke) to avoid screen reader noise.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why not just use `eval()` or `new Function()`?

Security (code injection if the input string is ever built from user text, URL or storage), strict CSP blocks it, and it doesn't understand our symbols × ÷ −. It also gives no control over errors (`5/0` gives `Infinity`, not a message). A 15-line evaluator is safer and explainable.

Q2. What if the interviewer wants a simple pocket calculator where `2 + 3 × 4 = 20`?

That's "immediate execution": keep `{ acc, pendingOp, current }` instead of tokens and compute `acc = apply(acc, pendingOp, current)` every time an operator is pressed. Simpler state, but no expression display or preview. That's why this was a clarifying question.

Q3. Why does `5 + ×` replace the operator instead of showing an error?

That's what every real calculator does and it's the forgiving choice: the user changed their mind. Since tokens must alternate number/operator, replacing keeps the invariant that `evaluate` relies on.

State design

Q4. Why store numbers as strings?

While typing, `'3.'`, `'3.0'` and `'-0'` are meaningful, but `Number()` turns them into `3`, `3`, `-0`, so the user could never type `3.05`. Strings keep exactly what was typed; we convert only inside `evaluate`.

Q5. Why not store `currentInput`, `result` and `preview` as separate state?

They are all derived from `tokens`. Storing them gives multiple sources of truth that can drift (preview showing an old value after backspace). Derive in render; it's cheap.

Q6. Why `useReducer` and not `useState`?

One key changes several fields at once (`=` sets tokens, evaluated, error). A reducer makes each transition atomic and puts every rule in one pure function, which I can unit test without rendering. With `useState` the rules spread across handlers.

Q7. Why do you need the `evaluated` flag? Can't you derive it?

No. `['14']` after typing `1`, `4` and `['14']` after `=` look identical, but the next digit behaves differently (append vs start fresh). That's real information about history, so it must be state.

Hard parts

Q8. How would you add brackets and powers?

Switch the evaluator to Dijkstra's shunting-yard: convert tokens to postfix (RPN) using an operator stack and a precedence table (`^` right-associative), then evaluate the RPN with a number stack. The reducer gets `(` / `)` actions with rules (auto-close on `=`, no `)` without an open `(`).

Q9. Your rounding hides errors. When is that not good enough?

For money or anything that must be exact (`0.1 × 3` is fine visually, but sums of many values drift). Then use a decimal library (`decimal.js`, `big.js`) or integer arithmetic in the smallest unit (cents). For a UI calculator,

12 significant digits matches what phone calculators show.

Q10. What happens on `999999999999999 × 999999999999999`?

The result is ~1e30, still finite; `toPrecision(12)` rounds it and the display shows `1e+30`. If it overflows to `Infinity` (e.g. `1e308 × 10`), `formatNumber` throws "Number too large" and the reducer shows the error state.

Q11. Why does pressing Enter on a focused button type twice, and how did you fix it?

A focused `<button>` turns Enter/Space into a click by default, and our window listener also sees Enter. `preventDefault()` on keydown cancels the button's default activation, so only our `equals` action runs. Another option: `onMouseDown={e => e.preventDefault()}` on buttons so they never take focus, but that hurts keyboard users, so I didn't.

Q12. Why is the `useEffect` dependency array empty? Isn't that a stale closure?

The handler only uses `dispatch` and constants. `dispatch` from `useReducer` is guaranteed stable, and the reducer always gets the latest state, so there's nothing stale. If I had used `useState` and read `tokens` inside the handler, it would be stale and I'd need a ref or a functional update.

Implementation

Q13. What if the page has a text input elsewhere? The global listener will steal its keys.

Bail out when `e.target` is an input/textarea/contenteditable, or attach the listener to the calculator element (`tabIndex={0}`, `onKeyDown`) so it only works when the calculator has focus. The global listener was an assumption for a standalone calculator.

Q14. How would you support multiple calculators on one page?

The component already owns its state, so two `<Calculator />`s work. Only the global keyboard listener conflicts: both would react. Fix as in Q13 (scope keys to the focused calculator).

Q15. How would you test this?

Table-driven unit tests on the reducer: feed a string of keys, assert tokens (I'd reuse the use-case table). `evaluate` / `formatNumber` tested alone. Then a few RTL tests: click `2 + 3 =` → display `5`; `fireEvent.keyDown(window, { key: 'Enter' })`; divide by zero shows the error.

```
const press = (keys: Action[]) => keys.reduce(calculatorReducer, initialState)
expect(press([d('2'), op('+'), d('3'), op('×'), d('4'), eq]).tokens).toEqual(['14'])
```

Q16. Is `aria-live` on the display a good idea?

It makes results audible, but announcing every keystroke can be noisy. Better: keep the display silent and have a visually hidden `aria-live="polite"` region that only receives the result after `=` or the error message.

Performance

Q17. Every key re-renders the whole calculator. Is that a problem?

No: 20 buttons and two text nodes render in well under a millisecond. If it mattered, `React.memo(Keypad)` would skip it entirely because `onPress` (`dispatch`) never changes. I wouldn't add memo without measuring.

Q18. The preview runs `evaluate` on every render. What if the expression had 10,000 tokens?

`evaluate` is O(n), so even 10k tokens is microseconds. If it ever got heavy, wrap it in `useMemo(() => getPreview(tokens), [tokens])`; the reducer returns a new array only when tokens change, so the memo key works.

Trade-offs and "what if"

Q19. What if they want history of past calculations, saved across reloads?

Add `history: { expr: string; result: string }[]` to the reducer, push on successful `=`. Persist with a `useEffect` writing to `localStorage` (try/catch, cap the length), and initialize with the lazy third argument of `useReducer`.

Q20. What if they want "repeat last operation" (press `=` again to keep adding 5)?

Store `lastOp: { op, operand }` when `=` runs. On `=` with `evaluated: true`, apply `lastOp` to the current result instead of doing nothing. One extra field, one extra branch.

Q21. Why not a library like `mathjs`?

It's ~100 kB+ for four operators, and the interviewer wants to see the logic. In a real product that needs scientific functions, `mathjs` (with its safe evaluate, not `eval`) is a fine choice.

Q22. Why put the key layout in an array instead of writing buttons in JSX?

The layout becomes data: reorder, add `√` or switch to a scientific layout without touching rendering. Every button goes through the same code path (same classes, same aria handling), so it's harder to forget an `aria-label` on one.